3.5 Alberi di ricomprimento minimo

Nel contesto di numerosi problemi applicativi — quali la progettazione di reti di calcolatori, linee telefoniche, reti elettriche o circuiti elettronici — un'istanza del problema è naturalmente modellata mediante un grafo connesso. In molte situazioni risulta utile estrarre da tale grafo una struttura ad albero che includa tutti i nodi originali, eliminando i cicli pur mantenendo la connettività. Un sottografo di questo tipo è detto *albero di ricoprimento (spanning tree)*. Per uno stesso grafo possono esistere molteplici alberi di ricoprimento. Nel caso di grafi non pesati, essi sono tutti equivalenti dal punto di vista del costo e uno qualsiasi può essere ottenuto mediante una semplice visita del grafo, memorizzando gli archi che conducono a nodi non ancora visitati. Al contrario, nel caso di grafi pesati diventa rilevante individuare un albero di ricoprimento *di costo minimo*, cioè uno spanning tree per il quale la somma dei pesi degli archi selezionati sia minima.

Def. 3.10 [Grafo pesato] Un grafo *pesato* è un grafo $G = (V, E)$ dove a ogni arco è associato un *peso* o *costo* denotato da una funzione $c\colon E \to \mathbb{R}$.

Il significato dei pesi in un grafo pesato dipende dall'applicazione: i pesi possono rappresentare distanze, tempi di percorrenza, costi di attività, ecc.

Def. 3.11 [Albero di ricoprimento minimo] Dato un grafo G connesso, non orientato e pesato, un *albero di ricoprimento minimo (minimum spanning tree)* di G è un albero di ricoprimento di G in cui la somma dei pesi degli archi è minima.

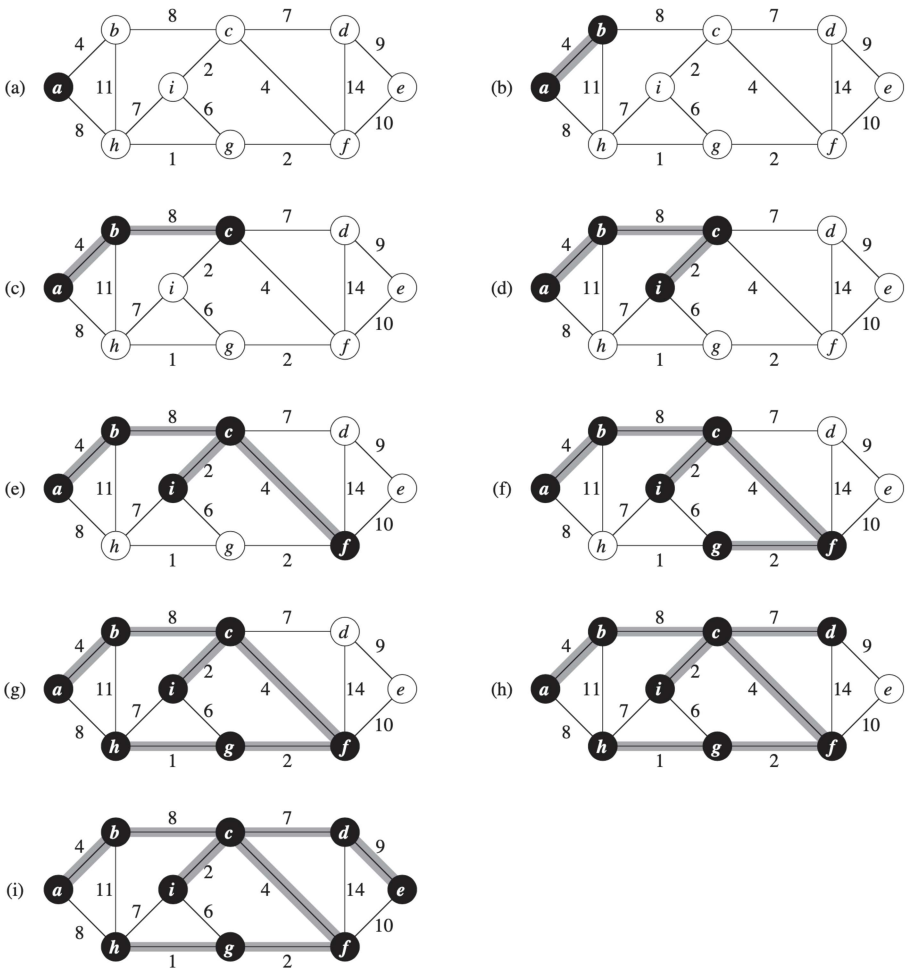
Un albero di ricoprimento minimo di G è quindi un sottografo di G tale che:

- sia un albero libero, ossia connesso e aciclico
- contenga tutti i nodi di G
- la somma dei pesi degli archi sia minima.

Nel seguito vediamo due algoritmi per il calcolo di un albero di ricoprimento minimo. Entrambi gli algoritmi utilizzano tecniche *greedy*: costruiscono l'albero incrementalmente e ogni volta che un arco e un nodo vengono aggiunti all'output la scelta è irrevocabile. Tuttavia, i due algoritmi procedono in modo diverso e richiedono strutture dati di supporto diverse: l'*algoritmo di Prim* effettua una visita del grafo , aggiungendo man mano gli archi meno costosi alla soluzione; l'*algoritmo di Kruskal* costruisce la soluzione bottom up, partendo con una foresta completamente sparsa e unendo man mano gli alberi di questa foresta attraverso archi di costo minimo.

3.5.1 Algoritmo di Prim

L'idea base di questo algoritmo è di visitare il grafo prendendo ogni volta, fra tutti i nodi adiacenti a quelli già inclusi nella soluzione (nodi neri), quello connesso a un nodo nero dall'arco di costo minimo, cioè si cerca il nodo "più vicino" all'albero già costruito. Questo algoritmo ha bisogno di utilizzare una *coda di priorità* per memorizzare la frangia di nodi adiacenti all'albero già costruito. La differenza fondamentale rispetto a una visita standard è che, mentre la visita standard visita i nodi della frangia nell'ordine in cui vengono forniti dalla coda (LIFO o FIFO secondo il tipo di visita), in questo caso è necessario selezionare ad ogni passo il nodo che minimizza l'operazione di espansione dell'albero. La distanza usata nella coda di priorità rappresenta la distanza di un nodo dall'albero già costruito e si assume infinita se il nodo non è adiacente all'albero stesso. La figura seguente (tratta dal CLRS) presenta un esempio di costruzione dell'albero con questo algoritmo, a partire dal nodo a, scelto in modo arbitrario.




```
Prim(G,s)
  for each (u nodo in G) marca u come non visitato //necessario
  for each (u nodo in G) dist[u] = ∞
  parent[s] = null; dist[s] = 0
  Q = heap vuoto
  for each (u nodo in G) Q.add(u, dist[u])
  while(Q non vuota)
    u = Q.getMin() //estraggo nodo a minima distanza dai neri
    marca u come visitato (nero)
    for each ((u,v) arco in G)
      if (v non visitato  &&  c_{u,v} < dist[v] )
        parent[v] = u; dist[v] = c_{u,v}
        Q.changePriority(v,dist[v]) //moveUp
```

Notiamo che occorre un controllo esplicito che i nodi adiacenti al nodo u estratto dalla coda non siano già stati visitati, perché non è detto che per un nodo v già visitato il test $c_{u,v}$ `< dist[v]` sia falso.

Correttezza. Sia T l'albero di nodi neri (non in `Q`) corrente. L'invariante è composta di due parti:

1. $T \subseteq MST$ per qualche MST albero di ricoprimento minimo di G
2. per ogni nodo $u \neq s$ in `Q`
   `dist`$[u]$ = costo minimo di un arco che collega u a un nodo nero.

(se questo arco non esiste consideriamo il costo minimo $= \infty$).
L'invariante vale all'inizio:

1. vale banalmente perché T è vuoto (non ci sono nodi neri)
2. vale banalmente perché non ci sono nodi neri e la distanza è per tutti infinito, tranne che per s.

L'invariante si mantiene.

1. Viene estratto dalla coda un nodo *u* con `dist`$[u]$ minima, cioè connesso a un nodo nero y da un arco di costo minimo tra tutti quelli che "attraversano la frontiera", cioè uniscono un nodo nero a un nodo non nero. L'arco (y, u) diventa quindi parte dell'albero di nodi neri corrente. Dimostriamo che aggiungendo tale arco si ha ancora un sottoalbero di un albero di ricoprimento minimo di G.

   Supponiamo per assurdo che aggiungendo (y, u) a T l'albero ottenuto non appartenga a nessun albero di ricoprimento minimo di G. Per l'invariante (1), $T \subseteq MST$ per un certo MST albero di ricoprimento minimo di G. MST, essendo un albero ricoprente, per definizione è un sottografo connesso che connette tutti i nodi di G: quindi in MST, se non c'è l'arco (y, u), ci deve essere un altro cammino da y a *u*. Questo cammino dovrà contenere un (primo) arco (x, z) che "attraversa la frontiera", ossia connette un nodo di T a un nodo non di T.

   Poiché MST è un albero, quindi ogni coppia di nodi è connessa da un unico cammino, se eliminiamo l'arco (x, z) otteniamo un grafo non connesso, costituito da due alberi (sottografi connessi aciclici), siano T_1 e T_2. Quindi se consideriamo il sottografo formato da T_1, T_2 e l'arco (y, u) otteniamo:

   - nuovamente un albero
   - che contiene tutti i nodi di G, quindi è un albero ricoprente
   - in cui la somma dei pesi degli archi è minore o uguale di quella di MST, in quanto differisce da quest'ultimo solo per l'arco (y, u) al posto dell'arco (x, z), e $c_{y,u} \leq c_{x,z}$ dato che $c_{y,u}$ è il minimo costo di un arco da un nodo nero a un nodo non nero.

   Quindi è un albero di ricoprimento minimo che contiene l'arco (y, u), contro l'ipotesi per assurdo.

2. L'insieme dei nodi neri risulta modificato per l'aggiunta di *u*, quindi occorre ripristinare l'invariante (2), controllando se per qualche nodo *v* non nero e adiacente a *u* l'arco (u, v) ha costo minore del precedente arco che univa *v* a un nodo nero, e in tal caso aggiornare l'arco e la distanza: questa operazione viene effettuata dalle ultime istruzioni dell'algoritmo (corpo dell'`if`).

Postcondizione: all'uscita dal ciclo tutti i nodi sono neri, quindi T connette tutti i nodi, cioè è un albero ricoprente di G. Per l'invariante (1), $T \subseteq MST$ per qualche MST albero di ricoprimento minimo, quindi $T = MST$.

Si noti che, per dimostrare che il ciclo mantiene l'invariante (1) è necessario dimostrare che il ciclo mantiene anche l'invariante (2).

Nota: L'algoritmo di Prim genera un albero radicato di radice s, ma l'albero di ricoprimento minimo è per definizione un albero non radicato, quindi è possibile ottenere lo stesso a partire da nodi diversi. In particolare, si può dimostrare che se i pesi degli archi sono tutti distinti l'albero di ricoprimento minimo è unico.

Complessità. Siano n e m rispettivamente il numero ci nodi e il numero di archi di G. Le operazioni dominanti sono quelle di accesso alla coda di priorità,. Assumendo un'implementazione tramite heap, il costo di ciascuna operazione è logaritmico nel numero di elementi in coda (al più n). La costruzione iniziale della coda (ciclo `for` iniziale) ha pertanto un costo $O(n \log n)$; in realtà, lo heap si potrebbe costruire in tempo lineare, ma questo non migliorerebbe la complessità dell'algoritmo. Il ciclo `while` viene eseguito n volte, perché i nodi sono messi in coda in precedenza e ad ogni ciclo ne viene tolto uno. Il costo globale di estrarre il minimo dalla coda ogni volta sarà quindi $O(n \log n)$. Il ciclo `for` interno al `while` viene eseguito complessivamente m volte, pertanto il costo totale di aggiornare la priorità dei nodi in coda è $O(m \log n)$. Sommando i contributi otteniamo un costo complessivo di $O((n + m) \log n)$.

3.5.2 Algoritmo di Kruskal

Questo algoritmo costruisce l'albero di ricoprimento minimo mantenendo una foresta, alla quale aggiunge a ogni iterazione un nuovo arco che unisce due sottoalberi distinti. Quindi l'algoritmo di Kruskal, a differenza di quello di Prim, non costruisce l'albero a partire da un nodo scelto come radice, ma come un insieme di archi che alla fine forma un grafo connesso aciclico, quindi un albero libero.

L'idea generale è di partire con una foresta costituita da tutti i nodi e nessun arco; quindi si attraversano gli archi del grafo per costo crescente e si aggiunge un arco alla foresta solo se questo connette due diverse componenti connesse della foresta. La figura seguente (tratta dal CLRS) mostra un esempio di costruzione dell'albero di ricoprimento minimo con questo algoritmo.